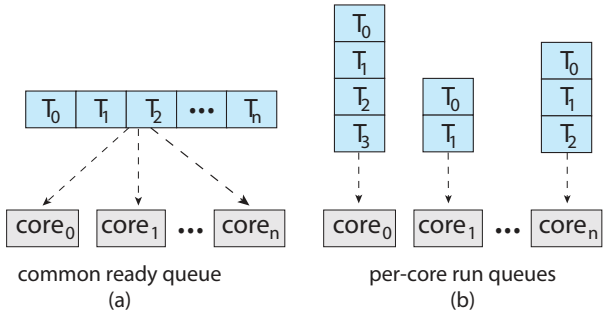
کرد، چرا که تمامی دسترسی‌ها به صف مستلزم مالیک قفل خواهند بود، و دسترسی صف مشترک احتمالاً یک گلوگاه کارایی به‌وجود خواهد آورد. گزینه‌ی دوم به هر پردازنده اجازه می‌دهد تا نخ‌ها را از صف اجرای خصوصی خودش زمان‌بندی نماید و لذا گرفتار مسائل کارایی مربوط به صف اجرای مشترک نمی‌شود. بنابراین، رویکرد بسیار رایج سیستم‌هایی است که SMP را پشتیبانی می‌کنند. به علاوه، همان‌گونه که در بخش ۵-۵-۴ بیان شد، داشتن صف‌های آماده‌ی خصوصی برای هر پردازنده می‌تواند به‌واقع باعث استفاده‌ی کارآمدتر از حافظه‌ی کَش شود. ملاحظاتی در رابطه با صف‌های اجرای هر پردازنده وجود دارد—بارکاری با اندازه‌های متغیر از موارد قابل توجه است. با این وجود، همان‌گونه که خواهیم دید، الگوریتم‌های موازنه می‌توانند برای برابرسازی بارهای کاری مابین تمامی پردازنده‌ها استفاده شوند.

تقریباً تمامی سیستم‌عامل‌های مدرن از SMP پشتیبانی می‌کنند، اعم از ویندوز، لینوکس، و macOS در کنار سیستم‌های موبایل شامل اندروید و iOS. در باقی این بخش، ملاحظات مربوط به سیستم‌های SMP را در زمان طراحی الگوریتم‌های زمان‌بندی پردازنده بحث می‌کنیم.



شکل ۵-۱۱ سازمان صف‌های آماده.

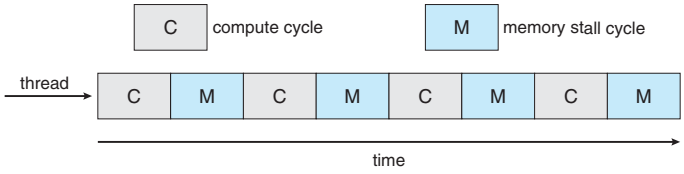
۵-۵-۲ پردازنده‌های چندهسته‌ای

سیستم‌های SMP سابق، با تدارک پردازنده‌های فیزیکی متعدد، اجرای موازی چندین پردازه را ممکن می‌ساختند. با این وجود، سخت‌افزار کامپیوترهای کنونی، هسته‌های پردازشی متعددی بر روی تراشه‌ی فیزیکی یکسانی می‌گذارند که نتیجه، یک **پردازنده‌ی چندهسته‌ای**[1] است. هر هسته یک مجموعه ثبات برای حفظ حالت معماریش دارد و لذا برای سیستم‌عامل یک پردازنده‌ی منطقی مجزا به حساب می‌آید. سیستم‌های SMP که از پردازنده‌های چندهسته‌ای استفاده می‌کنند، سریع‌ترند و نسبت به سیستم‌هایی که در آن، هر CPU تراشه‌ی فیزیکی خاص خود را دارند، مصرف انرژی پایین‌تری دارند.

پردازنده‌های چندهسته‌ای ممکن است ملاحظات زمانبندی را پیچیده‌تر کنند. حال ببینیم که چگونه این امر امکان‌پذیر است. محققان دریافته‌اند که هر زمان یک پردازنده حافظه را دسترسی می‌نماید، با انتظارکشی مهیا شدن داده‌ها، مقدار قابل توجهی زمان صرف می‌کند. این وضعیت

1. Multicore procesor

تحت عنوان **ایست حافظه**[1] اصولاً به این دلیل اتفاق می‌افتد که پردازنده‌های مدرن با سرعت‌های بسیار بیشتری از حافظه عمل می‌کنند. با این وجود، ایست حافظه می‌تواند به واسطه‌ی عدم توفیق کَش (دسترسی داده‌هایی که در حافظه‌ی کَش نیستند) نیز پیش بیاید. شکل ۵-۱۲، ایست حافظه را به نمایش می‌گذارد. در این حالت، پردازنده ممکن است تا ۵۰ درصد زمان خود را صرف انتظار مهیاشدن داده‌ها از حافظه نماید.



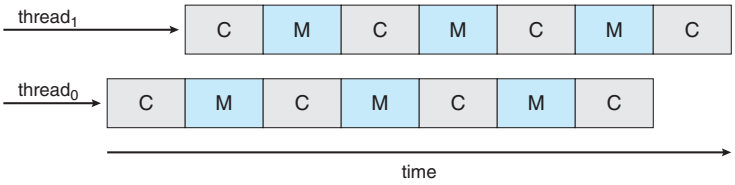
شکل ۵-۱۲ ایست حافظه.

بسیاری از طراحی‌های اخیر سخت‌افزار برای تعدیل این شرایط، هسته‌های پردازشی چندنخی را پیاده‌سازی کرده‌اند که در آن، به هر یک از هسته‌ها دو (یا چند) نخ سخت‌افزاری گمارده می‌شوند. با این شیوه، اگر یک نخ سخت‌افزاری به انتظار حافظه متوقف شود، هسته می‌تواند به نخ دیگر منتقل شود. شکل ۵-۱۳، هسته‌ی پردازشی دو-نخی را نشان می‌دهد که در آن، اجرای نخ ۰ و اجرای نخ ۱ یک در میان شده‌اند. از نقطه‌نظر یک سیستم‌عامل، هر نخ سخت‌افزاری حالت معماریش را، همچون اشاره‌گر دستورالعمل و مجموعه‌ثبات را نگه می‌دارد و لذا همچون یک CPU منطقی ظاهر می‌شود که برای اجرای یک نخ نرم‌افزاری در دسترس می‌باشد. این تکنیک—تحت عنوان **چندنخ‌سازی تراشه**[2] (**CMT**)—در شکل ۵-۱۴ نمایش داده می‌شود. در اینجا، پردازنده شامل چهار هسته‌ی پردازشی است که هر یک دو نخ سخت‌افزاری دارند. از نقطه‌نظر سیستم‌عامل، هشت CPU منطقی وجود دارد.

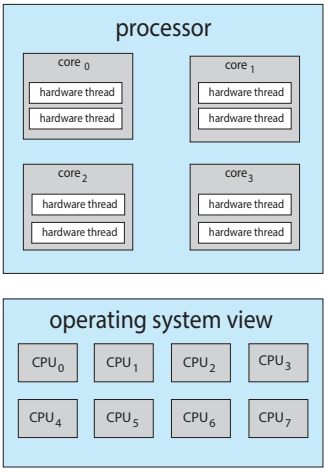
بنابراین، در یک سیستم دوهسته‌ای دونخی، چهار پردازنده‌ی منطقی در اختیار سیستم‌عامل هستند. پردازنده‌ی UltraSPARC T3، در هر تراشه، شانزده هسته و در هر هسته، هشت نخ سخت‌افزاری دارد. از دیدگاه سیستم‌عامل، ۱۲۸ پردازنده‌ی منطقی در دسترس هستند.

پردازنده‌های اینتل از تعبیر **ابرنخ‌سازی**[3] (که به **چندنخ‌سازی همزمان**[4] یا **SMT** نیز معروف است) استفاده می‌کنند تا گمارش نخ‌های سخت‌افزاری متعدد را به یک تک‌هسته‌ی پردازشی تبیین کنند. پردازنده‌های کنونی اینتل—همچون i7—از دو نخ به‌ازای هر هسته پشتیبانی می‌کنند، در حالی که پردازنده‌ی اوراکل‌اسپارک M7[5] از هشت نخ به‌ازای هر هسته، با هشت هسته به‌ازای هر پردازنده پشتیبانی می‌کند، لذا برای سیستم‌عامل ۶۴ CPU منطقی فراهم می‌شود.

1. Memory stall
2. Chip multithreading (CMT)
3. Hyper-threading
4. Simultaneous multithreading (SMT)
5. Oracle Sparc



شکل ۵-۱۳ سیستم چندهسته‌ای چندنخی.



شکل ۵-۱۴ چندنخ‌سازی تراشه.

در حالت کلی، دو روش برای چندنخی‌کردن یک هسته‌ی پردازشی وجود دارند که عبارتند از: چندنخی **دانه‌درشت**[1] و **دانه‌ریز**[2]. در چندنخی دانه‌درشت، یک نخ بر روی یک هسته اجرا می‌شود تا این که یک رویداد تأخیردار طولانی، همچون ایست حافظه پیش می‌آید. هسته بایستی به خاطر دیرکرد ناشی از رویداد تأخیردار طولانی به نخ دیگری منتقل شود تا اجرای آن را شروع کند. با این وجود، هزینه‌ی جابه‌جایی میان نخ‌ها بالاست، چون بایستی قبل از شروع اجرای نخ دیگر بر روی هسته‌ی پردازنده، خطلوله‌ی دستورالعمل خالی شود. همین که نخ جدید اجرا را شروع می‌کند، خطلوله را با دستورالعمل‌های خودش پر می‌کند. چندنخی دانه‌ریز (یا یک در میان) در سطح ریزدانه‌تری— نوعاً در حد دور یک دستورالعمل—میان نخ‌ها جابه‌جایی دارد. با این وجود، طرح معماری سیستم‌های دانه‌ریز برای جابه‌جایی نخ نیازمند مدارات منطقی است. در نتیجه، هزینه‌ی جابه‌جایی میان نخ‌ها پایین است.

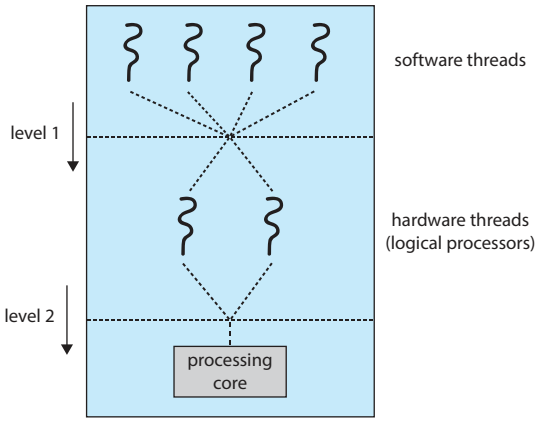
مهم است که توجه داشته باشید منابع هسته‌ی فیزیکی (همچون کَش‌ها و خطلوله‌ها) بایستی مابین نخ‌های سخت‌افزاری آن به‌اشتراک گذاشته شوند، و لذا یک هسته‌ی پردازشی می‌تواند در هر لحظه تنها یک نخ سخت‌افزاری را اجرا کند. در نتیجه، یک پردازنده‌ی چندهسته‌ای چندنخی

1. Coarse-grained
2. Fine-grained

به‌واقع مستلزم دو سطح متفاوت از زمان‌بندی است، آن‌گونه که در شکل ۵–۱۵ نشان داده شده که هسته‌ی پردازشی دونخی را نمایش می‌دهد.

در یک سطح، تصمیم‌های زمانبندی هستند که بایستی توسط سیستم‌عامل اتخاذ شوند مبنی بر این انتخاب که، کدام یک از نخ‌های نرم‌افزاری بر روی هر نخ سخت‌افزاری (CPU منطقی) اجرا شود. به‌خاطر تمامی اهداف عملی، این قبیل تصمیم‌ها به‌عنوان تمرکز اصلی این فصل می‌باشند. بنابراین، برای این سطح زمانبندی، سیستم‌عامل می‌تواند هر یک از الگوریتم‌های زمانبندی، اعم از مواردی که در بخش ۵–۳ بیان شده‌اند، را انتخاب کند.

سطح دوم زمانبندی مشخص می‌کند که چگونه هر یک از هسته‌ها تصمیم بگیرند که کدام نخ سخت‌افزاری را اجرا نمایند. برای استفاده در این شرایط چندین راهبرد وجود دارند. یک رویکرد، استفاده از الگوریتم نوبت–چرخشی ساده برای زمان‌بندی یک نخ سخت‌افزاری بر روی هسته‌ی پردازشی است. این رویکرد توسط پردازنده‌ی UltraSPARC T3 اقتباس شده است. رویکرد دیگری هست که توسط Intel Itanium استفاده می‌شود، یک پردازنده‌ی دوهسته‌ای با دو نخ به‌ازای هر هسته که توسط سخت‌افزار مدیریت می‌شود. به هر یک از نخ‌های سخت‌افزاری یک مقدار *فوریت*[1] پویا از ۰ تا ۷ نسبت داده می‌شود که، ۰ نشانگر پایین‌ترین و ۷ بالاترین فوریت است. در Itanium، پنج رویداد وجود دارند که جابه‌جایی یک نخ را باعث می‌شوند. زمانی‌که یکی از این رویدادها اتفاق می‌افتد، مدارات منطقی جابه‌جایی نخ، فوریت دو نخ را مقایسه کرده و نخی را که مقدار فوریت آن بالاتر است برای اجرا بر روی هسته‌ی پردازنده انتخاب می‌کنند.



شکل ۵–۱۵ دو سطح زمان‌بندی.

توجه کنید که دو سطح متفاوت زمان‌بندی نشان داده شده در شکل ۵–۱۵ الزاماً انحصار متقابل ندارند. در حقیقت، اگر زمان‌بند سیستم‌عامل (سطح اول) به اشتراک منابع پردازنده مطلع باشد، می‌تواند تصمیمات زمان‌بندی مؤثرتری بگیرد. به‌عنوان یک مثال، فرض کنید که یک CPU دو هسته‌ی پردازشی دارد، و هر هسته دارای دو نخ سخت‌افزاری است. اگر دو نخ نرم‌افزاری

1. Urgency

روی این سیستم اجرا شوند، می‌توانند بر روی هسته‌ی یکسانی و یا هسته‌های مجزا اجرا داشته باشند. اگر آن‌ها هر دو برای اجرا بر روی هسته‌ی یکسانی زمان‌بندی شوند، مجبور هستند منابع پردازنده را به‌اشتراک بگذارند و لذا احتمالاً نسبت به حالتی که بر روی هسته‌های مجزایی زمان‌بندی شوند، خیلی کندتر پیشروی خواهند داشت. اگر سیستم‌عامل به سطح اشتراک منبع پردازنده آگاه باشد، می‌تواند نخ‌های نرم‌افزاری را به روی پردازنده‌های منطقی که منابع را به‌اشتراک نمی‌گذارند، زمان‌بندی کند.

۳-۵-۵ تعادل بار

در سیستم‌های SMP مهم است که تعادل بارکاری میان تمامی پردازنده‌ها حفظ شود تا منافع داشتن پردازنده‌های متعدد به طور کامل فراهم شود. در غیر این صورت، یک یا چند پردازنده بیکار می‌نشینند، در حالی‌که بارکاری پردازنده‌های دیگر زیاد بوده و صف‌هایی از پردازه‌ها خواهند داشت که منتظر پردازنده‌های آنها هستند. **تعادل بار**[1] در صدد است تا بارکاری را میان پردازنده‌های یک سیستم SMP به صورت مساوی توزیع نماید. توجه به این نکته اهمیت دارد که تعادل بار فقط نوعاً در سیستم‌هایی ضرورت دارد که هر پردازنده، صف پردازه‌های آماده‌ی اجرای خاص خود را در اختیار دارد. اغلب در سیستم‌هایی که صف اجرا مشترک است، تعادل بار ضرورتی ندارد، چون به محض این که پردازنده‌ای بیکار شود، بلافاصله پردازه‌ای را از صف اجرای مشترک بر می‌دارد. با این وجود، توجه به این امر مهم است که در بیشتر سیستم‌عامل‌های معاصر که SMP را نیز پشتیبانی می‌کنند، هر پردازنده یک صف خصوصی از پردازه‌های آماده را برای خود دارد.

دو رویکرد کلی برای ایجاد تعادل بار وجود دارند که عبارتند از: **مهاجربری**[2] و **مهاجرکشی**[3]. در مهاجربری، وظیفه‌ای خاص به تناوب بار هر یک از پردازنده‌ها را بررسی می‌کند و—اگر متوجه شود که عدم تعادل وجود دارد—با جابه‌جایی (یا هل‌دادن) پردازه‌ها از پردازنده‌ی با بار زیاد به پردازنده‌های بیکار یا با مشغله‌ی کمتر، بار را به طور یکسان توزیع می‌نماید. مهاجرکشی زمانی اتفاق می‌افتد که پردازنده‌ی بیکاری، پردازه‌ای منتظر را از دوش یک پردازنده‌ی پرمشغله بر می‌دارد. مهاجربری و مهاجرکشی تناقضی با هم ندارند و در سیستم‌های تعادل بار در واقع به موازات هم پیاده‌سازی می‌شوند. برای مثال، زمانبند CFS لینوکس (که در بخش ۵-۷-۱ مطرح شد) و زمانبند ULE که برای سیستم‌های FreeBSD مهیا شده است، هر دو تکنیک را پیاده‌سازی می‌کنند.

مفهوم یک «بار متعادل» ممکن است معانی متفاوتی داشته باشد. یک دیدگاه از بار متعادل ممکن است به‌سادگی الزام کند که تمامی صف‌ها تقریباً تعداد نخ یکسانی داشته باشند. به‌روشی دیگر، تعامل ممکن است توزیع برابری از الویت نخ‌ها را در میان تمامی صف‌ها الزام کند. به‌علاوه، در شرایط خاصی، ممکن است هیچ یک از این راهبردها کافی نباشند. در حقیقت، ممکن است در

1. Load balancing
2. Push migration
3. Pull migration

مقابل اهداف الگوریتم زمان‌بندی قرار بگیرند. (ما بحث بیشتر راجع به این موضوع را به‌عنوان تمرین کنار می‌گذاریم.)

۵-۵-۴ خویشاوندی پردازنده

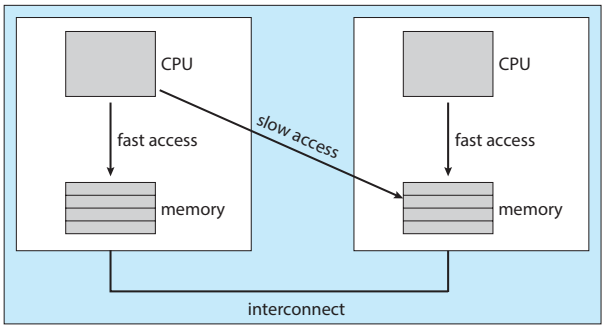
زمانی که یک نخ در پردازنده‌ای خاص اجرا شده است، فکر کنید که در حافظه‌ی کَش چه پیش می‌آید. داده‌هایی که اخیراً توسط نخ دستیابی شده‌اند در کَش پردازنده فراگیر می‌شوند. در نتیجه، دسترسی‌های بعدی حافظه که توسط نخ انجام می‌شوند در بیشتر موارد از طریق حافظه‌ی کَش (که به «کَش گرم» معروف است) قابل ارضا هستند. حال وضعیتی را در نظر بگیرید که نخ—برفرض، به‌دلیل تعادل بار—به پردازنده‌ای دیگر مهاجرت می‌کند. بایستی محتویات حافظه‌ی کَش پردازنده‌ی اول نامعتبر شده و حافظه‌ی کَش پردازنده‌ی دوم دومرتبه پر شود. به خاطر هزینه‌ی بالایی که در نامعتبرسازی و پرکردن مجدد حافظه‌های کَش وجود دارد، اکثر سیستم‌عامل‌ها با پشتیبانی SMP سعی می‌کنند از مهاجرت یک نخ از یک پردازنده به پردازنده‌ای دیگر جلوگیری کرده و در عوض تلاش می‌کنند تا نخ در حال اجرا را در همان پردازنده نگه بدارند و از مزیت کَش گرم بهره ببرند. این روال به **خویشاوندی پردازنده**[1] معروف است—یعنی، یک پردازه به پردازنده‌ای که فعلاً در آن اجرا می‌شود یک نوع وابستگی دارد.

دو راهبردی که در بخش ۵-۵-۱ جهت سازمان‌دهی صف نخ‌های در دسترس برای زمان‌بندی مطرح شدند عواقبی برای خویشاوندی پردازنده دارند. اگر ما رویکرد صف آماده‌ی مشترک را بپذیریم، یک نخ ممکن است توسط هر پردازنده‌ای برای اجرا انتخاب شود. بنابراین، اگر یک نخ روی یک پردازنده‌ی جدیدی زمان‌بندی شود، کَش آن پردازنده بایستی دومرتبه پر شود. با صف‌های آماده‌ی خصوصی برای هر پردازنده، یک نخ همواره بر روی پردازنده‌ی یکسانی زمان‌بندی می‌شود و لذا می‌تواند از محتویات یک کَش گرم بهره‌مند شود. اصولاً، صف‌های آماده‌ی به‌ازای هر پردازنده، خویشاوندی پردازنده را به‌رایگان فراهم می‌کنند!

خویشاوندی پردازنده اشکال متعددی دارد. زمانی‌که سیستم‌عامل برای حفظ پردازه‌ی در حال اجرا در همان پردازنده سیاستی دارد—اما نمی‌تواند تضمین کند که این سیاست را عملی خواهد ساخت—شرایطی داریم که به **خویشاوندی نرم** معروف است. در اینجا، سیستم‌عامل سعی دارد پردازه را بر روی یک پردازنده‌ی منفرد نگه بدارد، اما امکان مهاجرت پردازه میان پردازنده‌ها وجود دارد. در مقایسه، برخی سیستم‌ها جهت پشتیبانی **خویشاوندی سخت** نیز فراخوانی‌های سیستمی فراهم می‌کنند، با این کار یک پردازه اجازه می‌یابد، زیرمجموعه‌ای از پردازنده‌ها را که می‌تواند بر روی آنها اجرا شود مشخص کند. سیستم‌های بسیاری هر دوی خویشاوندی نرم و سخت را فراهم می‌کنند. برای مثال، لینوکس خویشاوندی نرم را پیاده‌سازی می‌کند، اما فراخوانی سیستمی sched_setaffinity() را دارد که با اجازه‌دادن به یک نخ به‌منظور مشخص‌کردن مجموعه‌ی CPUهایی که روی آن‌ها آمادگی اجرا را دارد، خویشاوندی سخت را نیز پشتیبانی می‌کند.

1. Processor affinity

معماری حافظه‌ی اصلی یک سیستم می‌تواند بر روی ملاحظات خویشاوندی پردازنده تأثیرگذار باشد. شکل ۵-۱۶، یک معماری با امکان دسترسی حافظه‌ی غیر یکشکل[1] (NUMA) را که در آنجا، دو تراشه‌ی پردازنده‌ی فیزیکی با CPU و حافظه‌ی محلی خاص خودشان وجود دارد. هرچند که اتصال بینابینی یک سیستم به تمامی CPUها در یک سیستم NUMA اجازه می‌دهد تا یک فضای آدرس فیزیکی را به‌اشتراک بگذارند، یک CPU به حافظه‌ی محلی خودش نسبت به حافظه‌ی محلی CPU دیگر دسترسی سریع‌تری دارد. اگر زمان‌بند CPU سیستم‌عامل و الگوریتم‌های مدیریت حافظه **آگاه به NUMA** باشند و با همدیگر کار کنند، در این صورت یک نخی که بر روی یک CPU خاصی زمان‌بندی شده است می‌تواند به نزدیک‌ترین حافظه‌ای تخصیص داده شود که CPU مقیم آن است، لذا سریع‌ترین دسترسی ممکن حافظه برای نخ فراهم می‌شود.



شکل ۵-۱۶ NUMA و زمانبندی پردازنده.

جالب است که غالباً تعادل بار با منافع خویشاوندی پردازنده در تضاد است. یعنی، نفع حفظ یک نخ در حال اجرا بر روی پردازنده‌ای یکسان این است که نخ می‌تواند از داده‌هایش که در حافظه‌ی کَش پردازنده قرار دارند بهره‌مند شود. متعادل‌ساختن بارها با انتقال یک نخ از یک پردازنده به دیگری، این نفع را از بین می‌برد. با تشابهی، مهاجرت یک نخ میان پردازنده‌ها ممکن است جریمه‌ای عاید سیستم‌های NUMA سازد، جایی که یک نخ ممکن است به یک پردازنده‌ای منتقل شود که زمان‌های دسترسی حافظه‌ی طولانی‌تری لازم دارد. به‌عبارت دیگر، یک کشمکش طبیعی میان تعادل بار و کمینه‌سازی زمان‌های دسترسی حافظه وجود دارد. بنابراین، الگوریتم‌های زمان‌بندی سیستم‌های NUMA چندهسته‌ای مدرن کاملاً پیچیده شده‌اند. در بخش ۵-۷-۱، ما الگوریتم زمان‌بندی CFS لینوکس را بررسی کرده و چگونگی موازنه‌ی این اهداف رقیب توسط این الگوریتم را مد نظر قرار می‌دهیم.

۵-۵-۵ چندپردازشی ناهمگن

در مثال‌هایی که تاکنون بحث کرده‌ایم، تمامی پردازنده‌ها با تعابیر قابلیت‌هایشان یکسان هستند، بنابراین به هر نخ اجازه می‌دهند تا بر روی هر هسته‌ی پردازشی اجرا شود. تنها اختلاف در

1. Non-uniform memory access (NUMA)

این است که زمان‌های دسترسی حافظه ممکن است بر اساس سیاست‌های تعادل بار و خویشاوندی پردازنده، همین‌طور بر روی سیستم‌های NUMA فرق داشته باشند.

هرچند که اکنون سیستم‌های موبایل شامل معماری‌های چندهسته‌ای هستند، برخی سیستم‌ها هم اکنون با استفاده از هسته‌ها طراحی می‌شوند که مجموعه‌ی دستورالعمل یکسانی را اجرا می‌کنند، هرچند با تعابیر سرعت ساعت و مدیریت انرژی، اعم از توانمندی تنظیم مصرف انرژی یک هسته در نقطه‌ی بیکاری هسته فرق دارند. این قبیل سیستم‌ها به **چندپردازشی ناهمگن**[1] (**HMP**) معروف هستند. توجه داشته باشید که این شکلی از چندپردازشی نامتقارن، آن‌گونه که در بخش ۵-۵-۱ مطرح شد نیست که وظایف سیستم و کاربر، هر دو می‌توانند بر روی هر هسته‌ای اجرا شوند. بلکه، هدف پشت سر HMP بهترکردن مدیریت مصرف انرژی با گمارش وظایف به هسته‌های خاص بر اساس نیازهای خاص وظیفه می‌باشد.

این نوع معماری در پردازنده‌های ARM که از آن پشتیبانی می‌کنند به **big.LITTLE** معروف است که هسته‌های **بزرگ** با کارایی بالاتر با هسته‌های **کوچک** کارآمدی انرژی ترکیب می‌شوند. هسته‌های **بزرگ** انرژی بیشتری مصرف می‌کنند و لذا فقط می‌بایست برای دوره‌های زمانی کوتاهی استفاده شوند. همین‌طور، هسته‌های **کوچک** انرژی کمتری مصرف می‌کنند و لذا می‌توانند برای دوره‌های طولانی‌تری استفاده شوند.

مزایای متعددی در این رویکرد وجود دارد. با ترکیب تعدادی از هسته‌های کندتر با نمونه‌های سریع‌تر، یک زمان‌بند پردازنده می‌تواند وظایفی را که کارایی بالایی لازم ندارند، اما ممکن است برای دوره‌های طولانی‌تری اجرا شوند، (همچون وظایف پس‌زمینه) به هسته‌های کوچک بگمارد، که با این کار به حفظ شارژ باتری کمک می‌کند. با تشابهی، برنامه‌های محاوره‌ای که انرژی پردازشی بیشتری لازم دارند، اما ممکن است برای دوره‌های کوتاه‌تری اجرا شوند، می‌توانند به هسته‌های بزرگ گمارده شوند. به‌علاوه، اگر دستگاه موبایل در مد صرفه‌جویی انرژی باشد، هسته‌های بزرگی که عطش انرژی دارند می‌توانند غیرفعال شده و سیستم می‌تواند صرفاً به هسته‌های کوچک با انرژی کارآمد متکی باشد. ویندوز ۱۰ با اجازه‌دادن به یک نخ در انتخاب یک سیاست زمان‌بندی که با نیازهای مدیریت انرژی آن سازگاری بهتری دارد از زمان‌بندی HMP پشتیبانی می‌کند.

۵-۶ زمانبندی بلادرنگ پردازنده

زمانبندی پردازنده برای سیستم‌های بلادرنگ مشمول ملاحظات خاصی است. در حالت کلی، می‌توانیم بین سیستم‌های بلادرنگ نرم و سیستم‌های بلادرنگ سخت تفاوت قایل شویم. **سیستم‌های بلادرنگ نرم** هیچ تضمینی نمی‌دهند که چه موقع، یک پردازه‌ی بلادرنگ بحرانی زمانبندی خواهد شد. فقط تضمین می‌کنند که پردازه نسبت به پردازه‌های غیربحرانی الویت خواهد گرفت. **سیستم‌های بلادرنگ سخت** نیازمندی‌های سخت‌گیرانه‌تری دارند. یک وظیفه بایستی تا ضرب‌الاجلی[2] که دارد سرویس داده شود؛ سرویس بعد از ضرب‌الاجل چنان است

1. Heterogeneous multiprocessing (HMP)
2. Deadline